

# More on Abstraction in Java

## Deadly Diamond of Death

---

Produced      Mairead Meagher  
by:            Dr. Siobhán Drohan  
                 Siobhán Roche

# **RECAP ON ABSTRACTION**

# Abstract vs Concrete

---

- Abstract
  - Implementation delayed
    - abstract method has no code
    - cannot instantiate an abstract class (it has, by definition “unfinished” methods)
- Concrete
  - Ready to go.
  - Everything up to now has been concrete.

# Abstract Methods

---

- Abstract methods have `abstract` in the signature.
- Abstract methods have no body.
  - ‘We promise to write this later. Every (concrete) subclass of this class will have this implemented in the subclass.’
- Abstract methods make the class abstract.
  - Think about why this is?

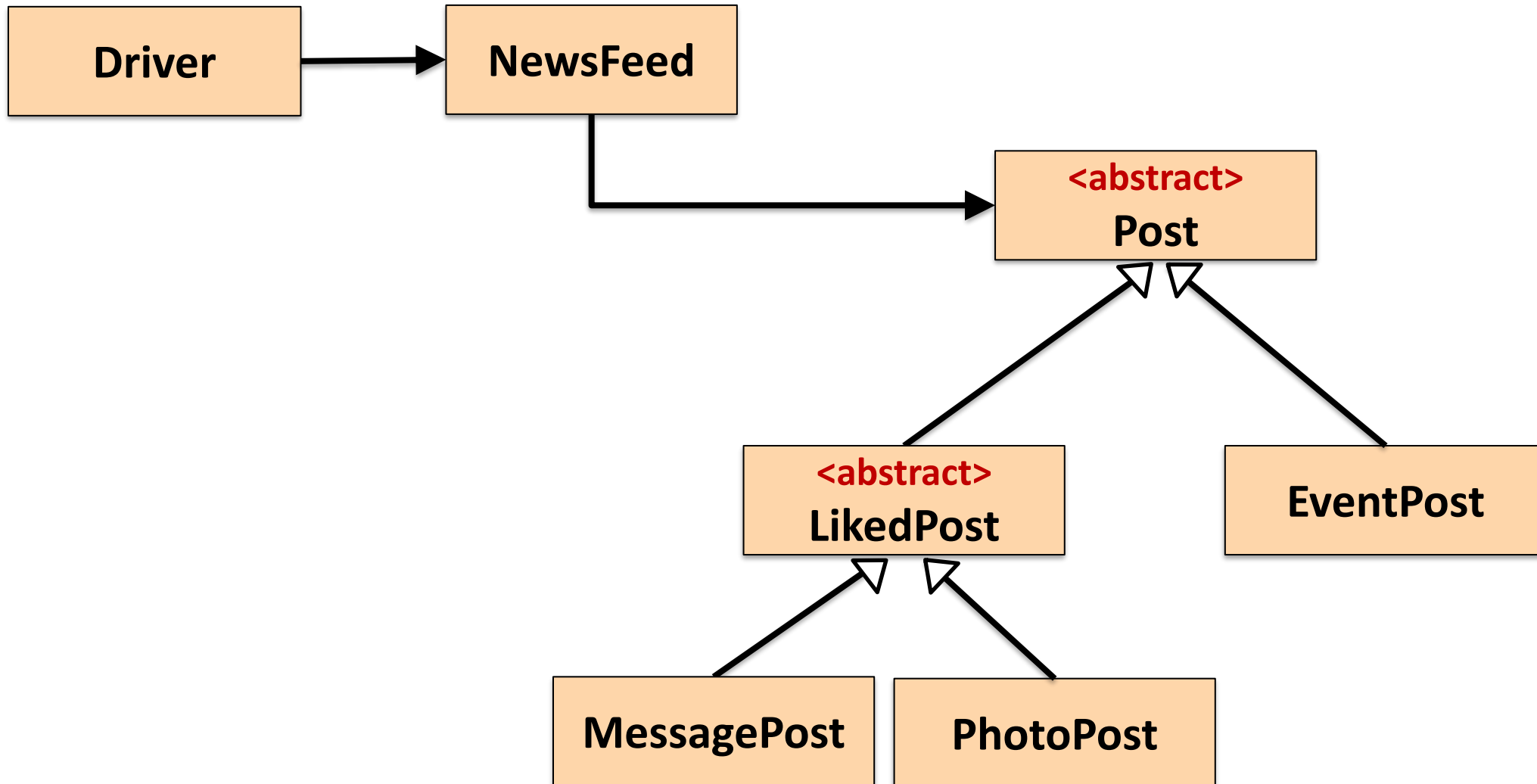
# Abstract Classes

---

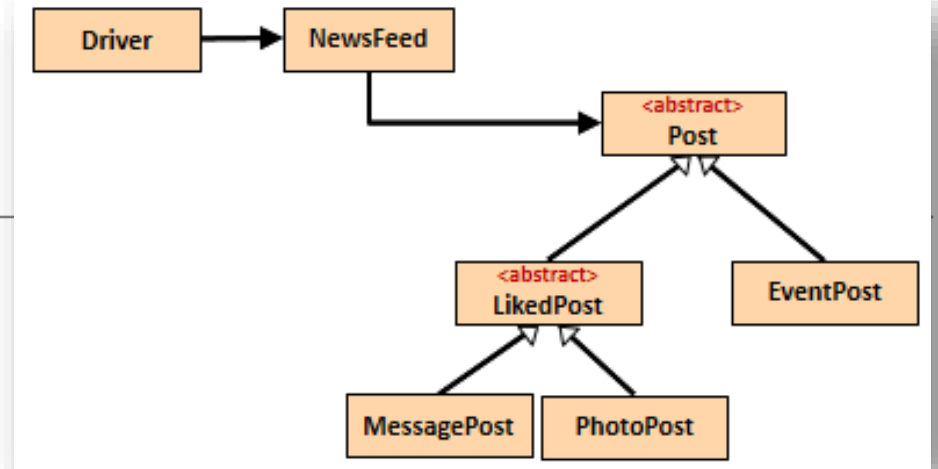
- An abstract class is a class that contains zero or more abstract methods.
- An class that has an abstract method must be declared abstract.
- Abstract classes cannot be instantiated.
- Abstract classes function as a “base” for subclasses.
  - abstract classes can be subclassed.
- Concrete subclasses complete the implementation.

## RECAP - Social Network V8.0 – Post and LikedPost Abstract

---



# RECAP - Syntax for abstract classes



//code omitted

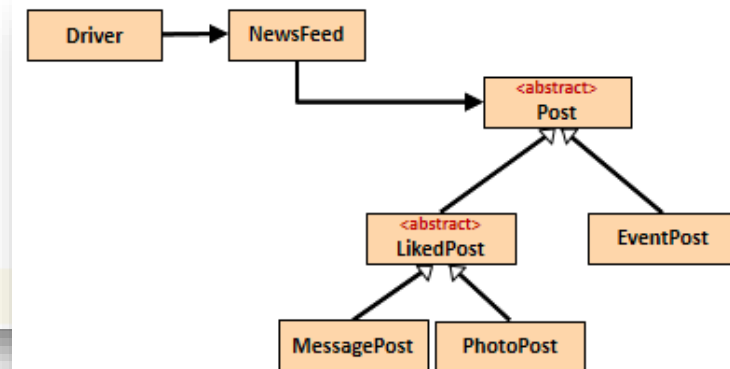
```
public abstract class Post {  
    private String author = "";  
  
    public Post(String author) {  
        this.author = Utilities.truncateString(author, 10);  
    }  
  
    //code omitted  
}
```

//code omitted

```
public abstract class LikedPost extends Post {  
    private int likes = 0;  
  
    public LikedPost(String author){  
        super(author);  
    }  
  
    //code omitted  
}
```

# displayCondensed() in Post

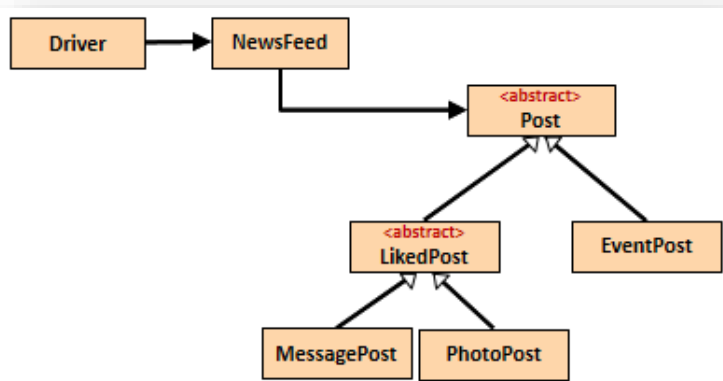
```
Post.java x
1 package models;
2
3 import utils.Utilities;
4
5 1 related problem
6 public abstract class Post {
7
8     private String author = "";
9
10    public Post(String author) { this.author = Utilities.truncateString(author, length: 10); }
11
12
13    public abstract String displayCondensed();
14
15    public String getAuthor() { return author; }
16
17
18    public void setAuthor(String author) {...}
19
20
21    public String display() { return (author + "\n"); }
22
23
24
25
26
27
28
29 }
```





## displayCondensed() in EventPost

The compiler is complaining that the concrete classes in the hierarchy have no implementation of **displayCondensed()**



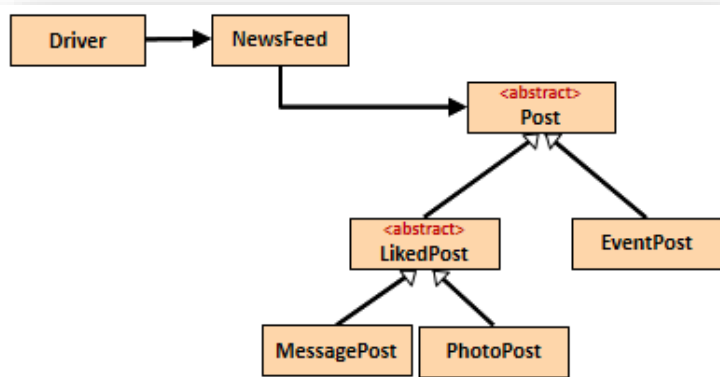
```
1 package models;
2
3 import utils.Utilities;
4
5 public class EventPost extends Post {
6
7
8
9
10
11
12 this.eventName = Utilities.truncateStri
13 setEventCost(eventCost);
14 }
```

The screenshot shows the **EventPost.java** file in an IDE. A red squiggly line under the **EventPost** class name indicates a compiler error. A context menu is open over the error, showing options: **Implement methods**, **Make 'EventPost' abstract**, **Create Test**, **Create subclass**, and **Make 'EventPost' package-private**. The **Implement methods** option is highlighted.

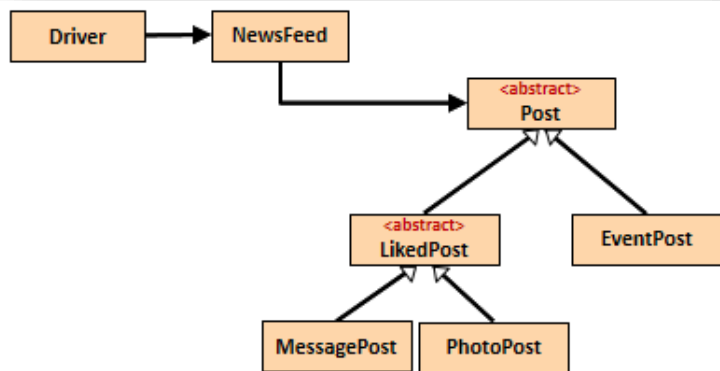
## displayCondensed() in EventPost



```
Post.java x EventPost.java x
1 package models;
2
3 import utils.Utilities;
4
5 public class EventPost extends Post {
6
7     private String eventName = "";
8     private double eventCost = 0;
9
10    public EventPost (String author, String eventName, double eventCost){...}
11
12    @Override
13    public String displayCondensed() {
14        return super.getAuthor() + ": Event(" + eventName + ", €" + eventCost + ").";
15    }
16
17    public String getEventName() { return eventName; }
18
19    public void setEventName(String eventName) {...}
20
21    public double getEventCost() { return eventCost; }
22
23    public void setEventCost(double eventCost) {...}
24
25    public String display() {...}
26
27 }
```



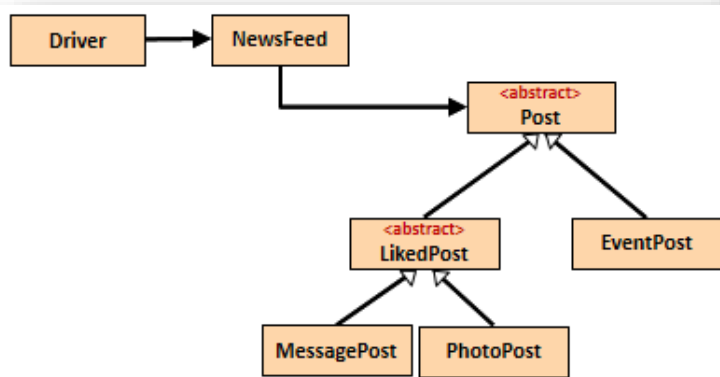
# displayCondensed() in PhotoPost



```
PhotoPost.java x
1 package models;
2
3 import utils.Utilities;
4
5 public class PhotoPost extends LikedPost{
6
7     private String caption = "";
8     private String filename = "";
9
10    public PhotoPost(String author, String caption, String filename) {...}
11
12    @Override
13    public String displayCondensed() {
14        return super.displayCondensed() + ": Photo(" + caption + ", " + filename + ")";
15    }
16
17    public String getCaption() { return caption; }
18
19    public void setCaption(String caption) {...}
20
21    public void setFilename(String filename) {...}
22
23    public String getFilename() { return filename; }
24
25    public String display() {...}
26
27 }
```

## displayCondensed() in MessagePost

```
MessagePost.java
1 package models;
2
3 import utils.Utilities;
4
5 public class MessagePost extends LikedPost{
6
7     private String message = "";
8
9     public MessagePost(String author, String message) {...}
10
11
12
13
14     @Override
15     public String displayCondensed() {
16         return super.displayCondensed() + ": Message(" + message + ")";
17     }
18
19     public String getMessage() { return message; }
20
21
22     public void setMessage(String message) {...}
23
24
25
26
27
28
29     public String display() {...}
30
31
32
33
34
35
36
37 }
38
```

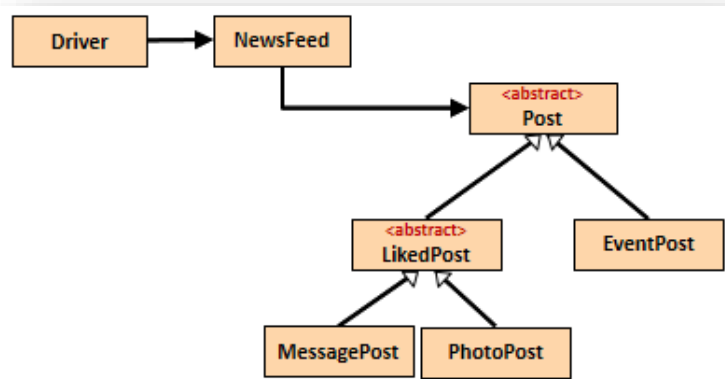


# displayCondensed() in NewsFeed

==>>1

List of All Posts are:

0: Siobhan (0 likes) : Message(My message is Hi There)  
1: Mairead (0 likes) : Photo(Hi all, photo7hello.jpg)  
2: Siobhan: Event(Coding Event, €5.0)



NewsFeed.java

```
1 package controllers;
2
3 import ...
4
12
13 public class NewsFeed {
14
15     private ArrayList<Post> posts;
16
17     public NewsFeed() { posts = new ArrayList<Post>(); }
18
19     public boolean addPost(Post post) { return posts.add(post); }
20
21     public String show() {
22         String str = "";
23
24         for(Post post: posts) {
25             str += posts.indexOf(post) + ": " + post.displayCondensed() + "\n";
26         }
27
28         if (str.isEmpty()){
29             return "No Posts";
30         }
31         else {
32             return str;
33         }
34     }
35 }
```

# **DEADLY DIAMOND OF DEATH**

# Deadly Diamond of Death!

---

- Recall that multiple inheritance is not allowed in Java.
- Any idea why the Java designers decided to not allow multiple inheritance?

# Deadly Diamond of Death!

---

- Recall that multiple inheritance is not allowed in Java.
- Any idea why the Java designers decided to not allow multiple inheritance?
- It is because of the Deadly Diamond of Death problem!



# Deadly Diamond of Death!

---

- This is easiest explained by example.
- Let's pretend that Java allows multiple inheritance and we will see really quickly what the Deadly Diamond of Death is!

# Deadly Diamond of Death - Example

---

- Suppose that we have an abstract super class, with an abstract method in it.

```
public abstract class AbstractSuperClass{  
    abstract void do();  
}
```

# Deadly Diamond of Death - Example

---

- Now two concrete classes extend this abstract super class.
- Each classes provides their own implementation of the abstract method defined in the super class.

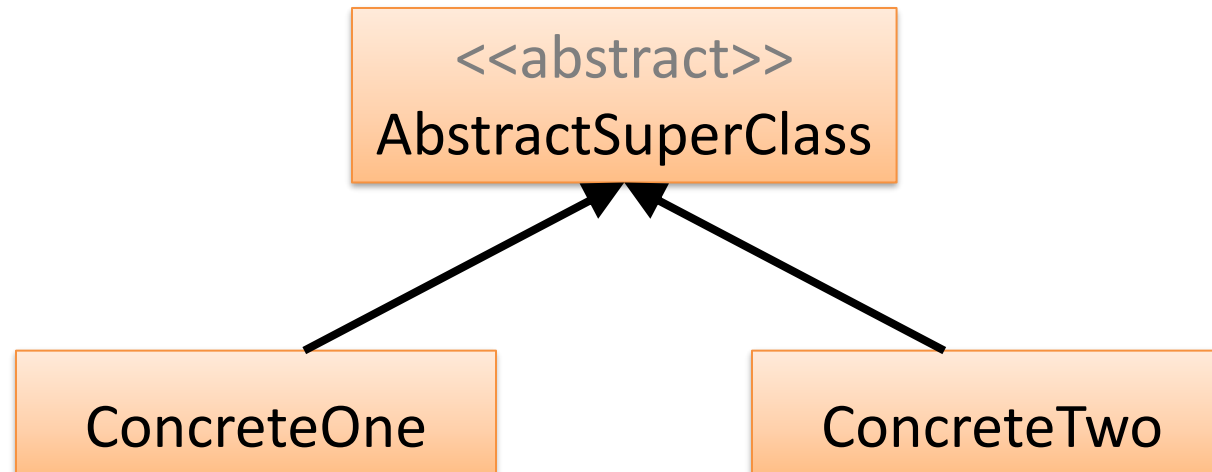
```
public class ConcreteOne extends AbstractSuperClass{  
    void do(){  
        System.out.println("I am testing multiple Inheritance");  
    }  
}
```

```
public class ConcreteTwo extends AbstractSuperClass{  
    void do(){  
        System.out.println("I will cause the Deadly Diamond of Death");  
    }  
}
```

# Deadly Diamond of Death - Example

---

- So far, our class diagram looks like this:



# Deadly Diamond of Death - Example

---

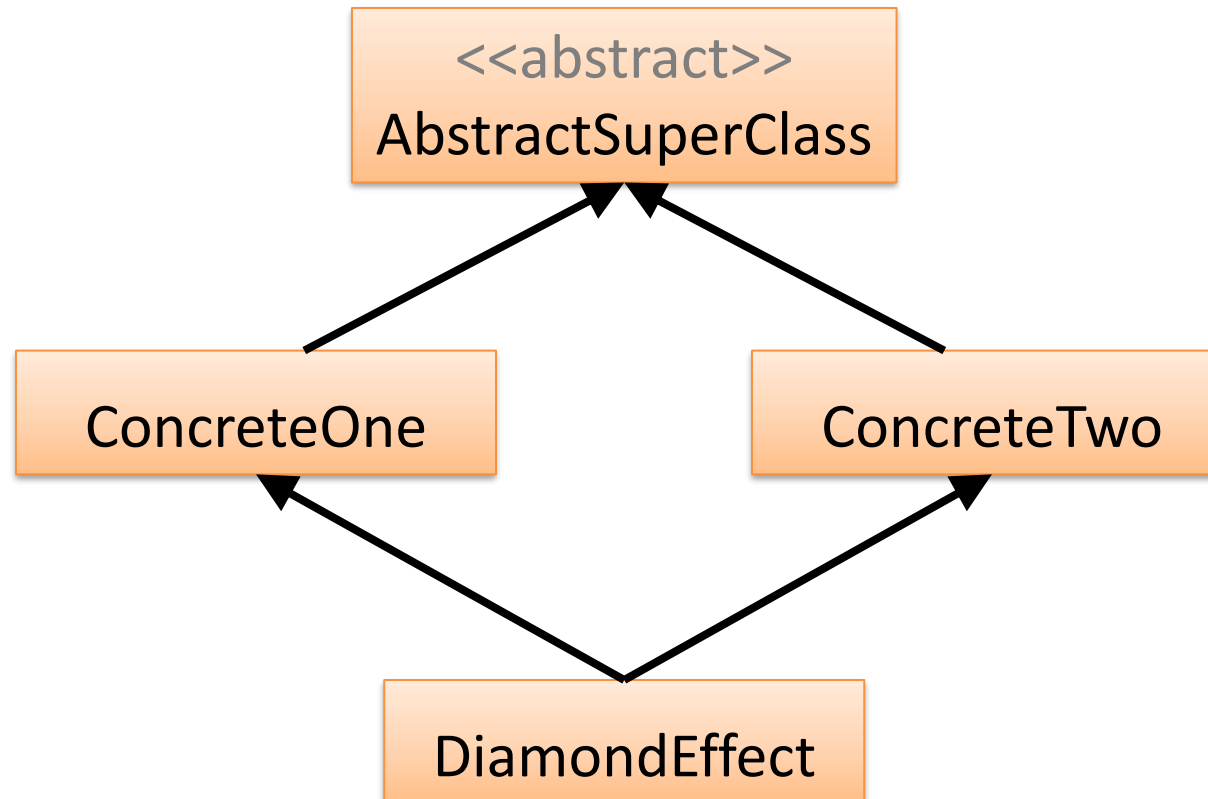
- Now a fourth class comes into picture which **extends** the above two concrete classes.

```
public class DiamondEffect extends ConcreteOne, ConcreteTwo{  
    //Some methods of this class  
}
```

# Deadly Diamond of Death - Example

---

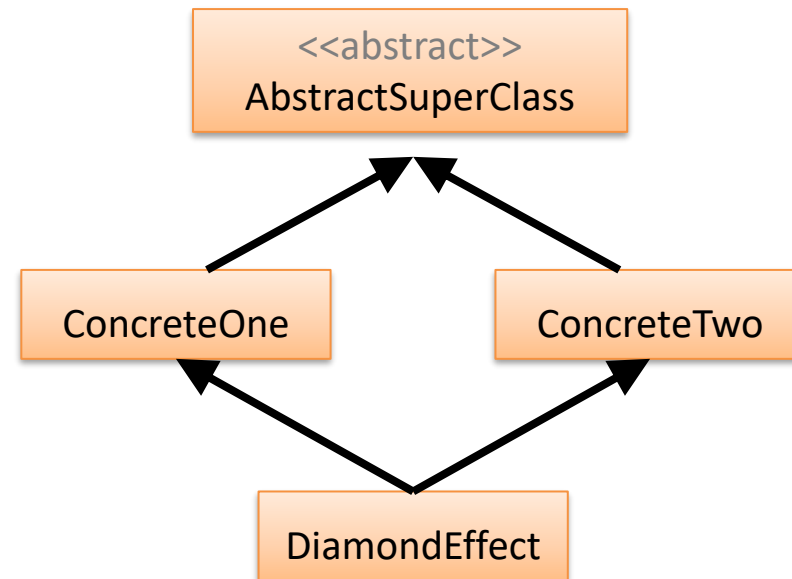
- Note that our class diagram is a diamond shape.



# Deadly Diamond of Death - Example

---

- The DiamondEffect class inherits all the methods of the parent classes.
- BUT we have a common method (`void do()`) in the two concrete classes, each with a different implementation.
- So which `void do()` implementation will be used for the DiamondEffect class as it inherits both these classes?



# Deadly Diamond of Death - Example

---

- Actually no one has got the answer to the above question, and so to avoid this sort of critical issue, **Java banned multiple inheritance**.
- The class diagram which is formed above is like that of a diamond, but with no solution or outcome, and so it is called Deadly Diamond of Death.



**Any  
Questions?**

